STEP 1 - First Look at the Data

Hotel Booking Cancellation Project | Data Exploration series

What this PDF is: a record of the very first thing we did with the data: load the file, check its size, its column types, a few raw rows, and how balanced the target is. Everything here was actually run on your machine, and the numbers below are the real outputs. Read it slowly; each section answers "what did we do", "why did we do it", and "what did we learn".

Libraries used in this step

Library	Version	Why we use it here
pandas	2.3.3	Loads the CSV into a DataFrame (a table in memory) and gives us .shape, .dtypes, .head(), .value_counts().
numpy	2.5.2	pandas is built on top of it. We do not call it directly in this step, but NaN (missing value) comes from numpy.
seaborn	0.13.2	Installed in this step but not used yet. It draws the correlation heatmap (later step).
matplotlib	3.11.1	The drawing engine under seaborn. Used later for charts.
scikit-learn	1.9.0	Models, splits and metrics. Not used until the modelling stage.
scipy	1.18.0	Statistics helpers. Later.

1. Why do a "first look" at all?

A common beginner mistake is to jump straight to modelling. A model will happily train on garbage and give you a score. A score you cannot explain is worthless in an interview. So the very first job is only to **look** and **not change anything**.

The first look answers four basic questions, each of which can stop us from wasting time later:

- **Is it the right file, and is it complete?** (shape)
- **What kind of thing is each column?** number, text, date (dtypes)
- **What does an actual row look like?** (head) Real values show things summaries hide.
- **How balanced is the thing we want to predict?** (target counts) This decides which metrics are even valid.

2. Environment check

Before loading data we confirmed three things about the setup:

Item	Result	Meaning
data/hotel_bookings.csv	Present, 16.9 MB	The dataset is in place. Small enough to load fully into memory on a laptop - this is what "medium-scale" means in practice.
pandas, numpy, sklearn, matplotlib, scipy, jupyter	Installed	Everything needed for exploration and modelling was already available.
seaborn	Was missing -> installed 0.13.2	Needed for the correlation heatmap. We asked you before installing because installing changes your environment.
.gitignore	Contains "data/"	The raw data stays out of git. Good practice: data files are large, can change, and may have licence limits. Code goes in git, data does not.

3. The code we ran

Saved in this same folder as **STEP_1_first_look.py**. You can run it yourself from the project root: `python data_exploration/STEP_1_First_Look/STEP_1_first_look.py`

```
import pandas as pd

pd.set_option('display.width', 200)        # wider printing so columns do not wrap
pd.set_option('display.max_columns', 50)   # show all columns instead of "..."

df = pd.read_csv('data/hotel_bookings.csv')   # load CSV into a DataFrame called df

print(df.shape)                 # (rows, columns)
print(df.dtypes)                # data type of every column
print(df.head(3).T)             # first 3 rows, transposed so columns run down the page
print(df['is_canceled'].value_counts())                 # counts of 0 and 1
print(df['is_canceled'].value_counts(normalize=True))   # same, as fractions
```

Line-by-line reasoning:

- **pd.read_csv** reads the file into a DataFrame. A DataFrame is a table: each row is one booking, each column is one attribute.
- **df.shape** returns (number of rows, number of columns). It is the cheapest sanity check there is.
- **df.dtypes** lists how pandas interpreted each column. pandas guesses from the contents, and guesses can be wrong.
- **df.head(3).T** shows the first 3 rows. The .T (transpose) flips it so the 32 column names run down the left side - with 32 columns, a normal head() is too wide to read.
- **value_counts()** counts each distinct value. With normalize=True it gives fractions instead of counts.

A small mistake worth remembering: my first attempt used the flag `python -I` (isolated mode, a safety option that ignores user-installed packages). pandas lives in the user packages, so Python said "No module named pandas". Lesson: this error does not always mean "not installed" - it can mean "installed somewhere Python is not looking". Interview-worthy debugging habit: check *where* Python looks before reinstalling.

4. Result 1 - Shape

```
SHAPE: (119390, 32)
```

119,390 rows and 32 columns. This matches the published paper (Antonio, Almeida and Nunes), so we have the correct, complete file. **Each row is one booking** (one reservation), not one guest and not one night. That distinction matters: the same guest can appear in several rows, and one row can cover many nights.

Why this matters: if the shape were, say, 60,000 rows, we would suspect a broken download. Also, 119K rows is comfortable for classical ML (trees, boosting, logistic regression) and large enough that a few percent of weird rows will not break a model - but it is small enough that we can inspect problems by hand.

5. Result 2 - Column data types

Here are all 32 columns with the type pandas assigned, and what that type tells us:

Column	pandas type	What it means / what the type tells us
hotel	object (text)	City Hotel or Resort Hotel. Categorical.
is_canceled	int64	TARGET. 0 = not cancelled, 1 = cancelled.

Column	pandas type	What it means / what the type tells us
lead_time	int64	Days between booking and arrival.
arrival_date_year	int64	2015, 2016 or 2017.
arrival_date_month	object	Month NAME as text (July...). Needs ordering/encoding later.
arrival_date_week_number	int64	Week of the year.
arrival_date_day_of_month	int64	Day number.
stays_in_weekend_nights	int64	Weekend nights in the stay.
stays_in_week_nights	int64	Weekday nights in the stay.
adults / babies	int64	Guest counts.
children	float64	Guest count, but float -> it has missing values (see below).
meal	object	Meal plan code (BB, HB, FB, SC, Undefined).
country	object	Country code (ISO-style, e.g. PRT = Portugal).
market_segment / distribution_channel	object	How the booking came in (online agency, direct, corporate...).
is_repeated_guest	int64	1 if the guest stayed before.
previous_cancellations / previous_bookings_not_canceled	int64	Guest history counts.
reserved_room_type / assigned_room_type	object	Room booked vs room actually given.
booking_changes	int64	How many times the booking was modified.
deposit_type	object	No Deposit / Non Refund / Refundable.
agent	float64	Travel agent ID. Float -> has missing values.
company	float64	Company ID. Float -> has missing values (mostly missing, as you knew).
days_in_waiting_list	int64	Days before booking was confirmed.
customer_type	object	Transient, Contract, Group, Transient-party.
adr	float64	Average Daily Rate. Regression target. Continuous number.
required_car_parking_spaces / total_of_special_requests	int64	Counts.
reservation_status	object	Check-Out / Canceled / No-Show. LEAKS the target.
reservation_status_date	object	Date stored as TEXT. Also leaks the target.

Three dtype lessons

- **object means text.** Models cannot use text directly, so every object column will need encoding (turning categories into numbers). How we encode depends on the column - that will be a separate documented decision.
- **Float where you expect whole numbers is a clue.** children, agent and company are float64 although they are counts or IDs. pandas stores missing values as NaN, and NaN only exists in float columns, so one missing value forces the whole column to float. In the rows we saw, agent and company show NaN directly.
- **Dates stored as text.** reservation_status_date is object. If we ever used it, we would convert it with pd.to_datetime first. (We will most likely drop it because of leakage - see section 7.)

6. Result 3 - The first rows (what real data looks like)

We printed the first 3 rows transposed. All three are Resort Hotel bookings for 1 July 2015 and all three were not cancelled. Key values:

Column	Row 0	Row 1	Row 2
lead_time	342	737	7
adults	2	2	1
stays (weekend / week nights)	0 / 0	0 / 0	0 / 1
reserved_room_type	C	C	A
assigned_room_type	C	C	C
booking_changes	3	4	0
agent / company	NaN / NaN	NaN / NaN	NaN / NaN
adr	0.0	0.0	75.0
reservation_status	Check-Out	Check-Out	Check-Out
reservation_status_date	2015-07-01	2015-07-01	2015-07-02

Things we noticed (each one is a question to investigate later, not a conclusion yet)

- **lead_time = 737 (row 1).** About two years between booking and arrival. This fits your hypothesis (long lead time means higher cancellation risk) but it is also an extreme value. Is it a genuine early booking or a data oddity? We will check the distribution of lead_time and see how common values this large are.
- **adr = 0.0 for rows 0 and 1**, even though the guests checked out. A price of zero for a real stay is strange. Possible explanations: complimentary stay, internal/staff booking, a special booking type, or missing data recorded as 0. We must not guess - we will look at which rows have adr = 0 and what else is true of them.
- **Rows 0 and 1 have 0 stay nights** (0 weekend + 0 week nights). A stay of zero nights is another oddity (same-day or data entry). Worth counting later.
- **reserved_room_type = A but assigned_room_type = C (row 2).** The room the guest got differs from the room booked. The assigned room is decided at or near check-in, so it may not be known when a prediction is needed. This is a **leakage candidate**.
- **reservation_status = Check-Out on all three rows, all with is_canceled = 0.** The status states the outcome. See section 7.
- **agent and company are NaN on all three rows.** Consistent with the "mostly missing" issue you already knew about.

7. Result 4 - The target and class imbalance

```
is_canceled
0    75166
1    44224

is_canceled
0    0.6296
1    0.3704
```

Class	Meaning	Count	Share
0	Not cancelled	75,166	62.96%
1	Cancelled	44,224	37.04%
Total		119,390	100%

Is 37% imbalanced? Yes, moderately. It is not extreme like fraud detection (often under 1%), but the classes are clearly unequal. The more important effect is on **metrics**.

The "always say no" baseline

Imagine a useless model that predicts "not cancelled" for every single booking. Its accuracy would be:

```
accuracy = correct predictions / all predictions
         = 75,166 / 119,390
         = 0.6296  ->  62.96%
```

It scores almost 63% while catching **zero** of the 44,224 cancellations - and catching cancellations is the only reason the model exists. This is called the **majority-class baseline** (or "dummy classifier"). It gives us a floor: any real model must clearly beat 62.96% accuracy, and accuracy alone cannot even show whether it does so in a useful way.

This is the start of your answer to the question "why ROC-AUC and not accuracy?" Accuracy rewards getting the big class right and can look good while the model is useless on the class you care about. The full justification (with formulas for recall, precision, F1, TPR, FPR, ROC and AUC) gets its own dedicated PDF in the metrics stage.

Why leakage is already visible

Data leakage means a feature contains information that would not be available at the moment we need to make the prediction, or that directly encodes the answer. The model then looks brilliant in testing and fails in the real world.

- **reservation_status** takes the values Check-Out, Canceled, No-Show. It is literally the outcome written in words. Using it to predict is_canceled is like predicting an exam result from the already-printed marksheet.
- **reservation_status_date** is the date that status was set, so it also depends on the outcome.
- **Test to apply to every column:** "At the moment a booking is made (or whenever we run the prediction), would I truly know this value?" If no, it cannot be a feature. Next step we will apply this test to all 32 columns.

8. Summary: what we learned in Step 1

Check	Finding	Consequence
Shape	119,390 x 32	Correct file; one row = one booking.
Types	12 text-type columns, 3 float columns with gaps, rest integers	Encoding needed for text; float columns hide missing values.
Target	62.96% / 37.04%	Moderately imbalanced; accuracy is misleading; baseline to beat = 62.96%.
Leakage	reservation_status and its date clearly leak	Must be excluded from features. assigned_room_type is a suspect.
Oddities	lead_time 737, adr 0, zero-night stays, NaN agent/company	Each needs a documented investigation, not a blind fix.

9. Interview questions from this step (with model answers)

Q1. Your target is 37% positive. Is that imbalanced? How do you detect it and what can you do?

Answer: Detect it with value_counts(normalize=True). 37/63 is moderately imbalanced. Options: choose metrics that do not depend on class ratio (recall, precision, F1, ROC-AUC, PR-AUC), use stratified train/test splits so both sets keep the 37/63 ratio, use class weights, or resample (oversample minority / undersample majority). With 37% we usually start with proper metrics and stratification before touching resampling.

Q2. A model gets 63% accuracy on this data. Is it good?

Answer: Not necessarily. 62.96% is exactly what a model gets by always predicting "not cancelled". Accuracy of 63% could mean the model learned nothing. We compare against this baseline and look at recall/precision on the cancelled class.

Q3. Why did pandas make children, agent and company float64?

Answer: These columns contain missing values stored as NaN. NaN is a floating-point concept, so a column of integers with even one NaN becomes float. It is a hint to check for missing data. (Newer pandas "nullable" integer type Int64 can hold missing values without becoming float.)

Q4. What is data leakage? Give an example from your project.

Answer: Leakage is when a feature carries information not available at prediction time or encodes the target. Here, reservation_status contains "Canceled" or "No-Show" for exactly the rows where is_canceled = 1 (confirmed with a cross-table in Step 2), so a model using it would score near perfect and be useless in production, because at booking time the status does not exist yet.

Q5. A price column has 0 for some rows. What do you do?

Answer: Investigate before acting. Count them, look at other columns for those rows (nights, segment, status), and decide: genuine free stay (keep, maybe flag), error/missing-coded-as-zero (impute or drop), or outlier. Document the decision and the reason. Never silently delete.

Q6. Why look at the data before modelling?

Answer: Models cannot tell sense from nonsense: they will train on duplicated rows, leaked columns, impossible values and still output a score. Only inspection reveals these problems, and a score built on them cannot be defended.

Q7. What does one row represent in this dataset and why does it matter?

Answer: One booking. It matters because the same guest or same hotel stay-period can appear several times, so rows are not fully independent - relevant when we split data for validation and when we interpret counts.

10. Self-check (answer these in your own words before Step 2)

- Why did we only look and not clean in this step?
- What accuracy does a "never cancelled" model get here, and why does that make accuracy a poor sole metric?
- Why are children, agent and company floats?
- Name two columns that leak the target and explain the test you used to decide.
- What would worry you about a row with lead_time = 737 and adr = 0?

What comes next: STEP 2 - Data Dictionary

We go through all 32 columns one by one: meaning, type, missing count, number of unique values, and a clear verdict - **usable feature, needs care, or leakage** - with the reason written down.